

Meteora Liquidity Distributor

Smart Contract Security Assessment

October 2025

Prepared for:

Meteora

Prepared by:

Offside Labs

Ronny Xing





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	6
4.1	Different Types of Activation Point Comparison Allows Past Activation	6
4.2	Informational and Undetermined Issues	6
5	Disclaimer	8



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of *Liquidity Distributor* smart contracts, starting on September 29th, 2025, and concluding on September 30th, 2025.

Project Overview

The Liquidity Distributor is a Solana program developed by Meteora that enables efficient distribution of DAMM v2 liquidity positions using Merkle tree verification. The program allows administrators to create distributors containing liquidity positions that users can claim in portions according to a pre-computed Merkle root.

The protocol uses a claim-based model where users verify their allocation through Merkle proofs, reducing upfront costs by shifting execution costs to claimants. When a user claims, the program verifies their proof against the Merkle root, splits the allocated liquidity shares from the distributor's position, and adds them directly to the user's own DAMM v2 liquidity position.

Audit Scope

The assessment scope contains mainly the smart contracts of the liquidity distributor program for the *Liquidity Distributor* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Liquidity Distributor
 - Codebase: <https://github.com/MeteoraAg/liquidity-distributor>
 - Commit Hash: d1f9bc43e60e0bb4d16058c737367639b017b162
- DAMM V2
 - Codebase: [PR-117](#)
 - Commit Hash: 7d7931d3447a8e2ae6a1c1583244f59e2f2312f6

We listed the files we have audited below:

- Liquidity Distributor
 - programs/liquidity-distributor/src/*.rs
 - verify/src/lib.rs
 - merkle-tree/src/*.rs
- DAMM V2
 - programs/cp-amm/src/constants.rs
 - programs/cp-amm/src/event.rs
 - programs/cp-amm/src/instructions/ix_split_position.rs
 - programs/cp-amm/src/instructions/ix_split_position2.rs
 - programs/cp-amm/src/instructions/mod.rs



- programs/cp-amm/src/lib.rs
- programs/cp-amm/src/math/utils_math.rs
- programs/cp-amm/src/state/pool.rs
- programs/cp-amm/src/state/position.rs

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 0 medium issue
- 1 low issue
- 2 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Different Types of Activation Point Comparison Allows Past Activation	Low	Fixed
02	Unnecessary Mutability on Account Constraints in Instruction Definitions	Informational	Fixed
03	Potential Corruption of Accounting Records	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 Different Types of Activation Point Comparison Allows Past Activation

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

The instruction `set_activation_point` uses the following check to ensure the current distributor is never activated until now.

```
27     let current_timestamp = Clock::get()?.unix_timestamp as u64;
28
29     require!(
30         distributor.activation_point >= current_timestamp,
31         crate::error::ErrorCode::InvalidActivationPoint
32     );
```

[programs/liquidity-distributor/src/instructions/
set_activation_point.rs#L27-L32](#)

However, it directly uses `unix_timestamp` as the type for `activation_point` without considering distributors where the `activation_type` is set to `ActivationType::Slot`.

Impact

The `activation_point` of an already activated distributor may be incorrectly reset.

Recommendations

Use `MerkleDistributor.get_activation_handler`.

Mitigation Review Log

Fixed in [PR-20](#).

4.2 Informational and Undetermined Issues

Unnecessary Mutability on Account Constraints in Instruction Definitions

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Data Validation



The following anchor constraints mark accounts such as `admin` and `new_admin` as `mut` even though the handlers never modify their data, resulting in unnecessary write locks and increased transaction cost.

1. [programs/liquidity-distributor/src/instructions/set_admin.rs#L17-L24](#)
2. [programs/liquidity-distributor/src/instructions/set_activation_point.rs#L15-L15](#)
3. [programs/liquidity-distributor/src/instructions/set_clawback_receiver.rs#L21-L21](#)

Potential Corruption of Accounting Records

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

Instruction `close_claim_status` is an endpoint for testing that can close `ClaimStatus` accounts by `admin`.

Closed `ClaimStatus` accounts can be recreated via the `new_claim` instruction, allowing a previously claimed amount to be claimed again and incrementing `num_nodes_claimed`. This corrupts accounting records and jeopardizes subsequent users' claims.

Recommendations: Add a `localnet` feature flag to the `close_distributor` and `close_claim_status` instructions and enforce that they can only be executed on non production clusters. This prevents accidental recreation of closed claim status records in live environments.

And same goes for the `ClaimStatus.admin` field.



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

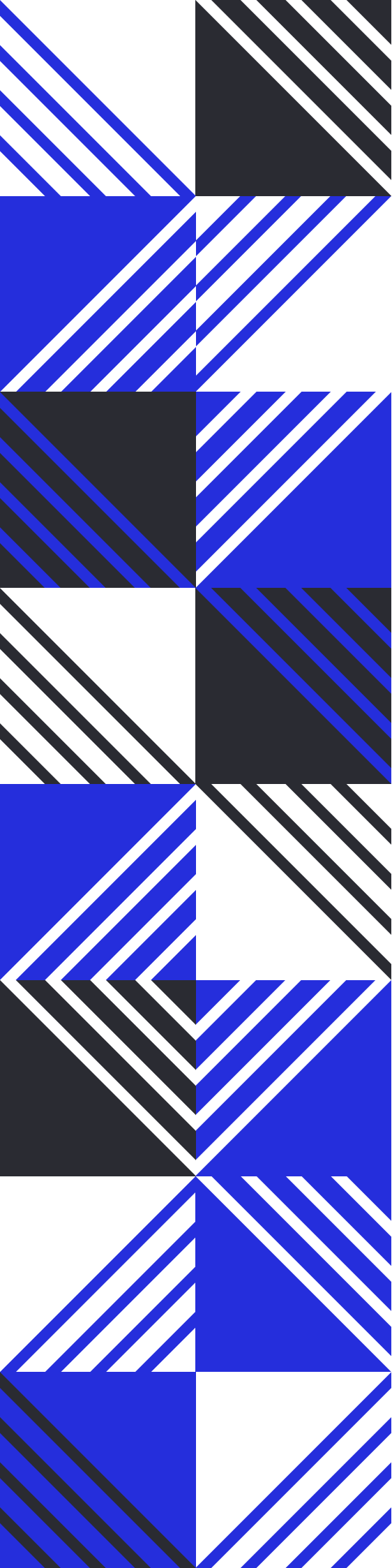
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs